



Sistema de Gestión de Logística para Distribución de Medicamentos Críticos

Integrantes: Martin Lagos

Diego Clemente

Mayra Salinas

Asignatura: Estructura de Datos

Fecha: 29 – Junio – 2026

Docente: Giordano Castro Villaroel

Introducción

El presente informe técnico documenta el desarrollo de un sistema de gestión logística de pedidos, elaborado en el marco de la asignatura de Estructuras de Datos y Algoritmos. La solución implementada da respuesta a un caso real planteado en la evaluación, cuyo objetivo es administrar eficientemente los pedidos de medicamentos realizados por distintas instituciones de salud (hospitales, clínicas, CESFAM y farmacias), asegurando el seguimiento de cada solicitud, la priorización de entregas y la detección oportuna de posibles retrasos.

El sistema ha sido diseñado e implementado en lenguaje Java, integrando los principales conceptos y técnicas estudiados durante el semestre: estructuras de datos lineales y no lineales, algoritmos de búsqueda y ordenamiento, recursividad, metodología Divide and Conquer, y análisis de complejidad computacional mediante notación Big O. Asimismo, se han aplicado buenas prácticas de programación, documentación del código y control de versiones con Git, cumpliendo con los requerimientos funcionales y técnicos establecidos en la prueba.

Descripción del Problema

La empresa para la que desarrollamos el sistema recibe muchos pedidos de medicamentos desde hospitales, clínicas, CESFAM y farmacias. Cada pedido tiene información importante como un código único, la institución que lo solicita, el tipo de medicamento, la cantidad, la prioridad (alta, media o baja), la fecha de entrega máxima y el estado actual.

El problema principal es que, sin un sistema automatizado, se hace difícil saber cuáles son los pedidos más urgentes y cuáles se están demorando. Muchas veces los pedidos con prioridad alta se atrasan porque no hay una forma clara de visualizarlos primero, y tampoco se detecta a tiempo si un pedido está cerca de vencer. Esto puede afectar la atención de los pacientes, ya que los medicamentos no llegan a tiempo.

Para solucionar esto, el sistema debe poder:

- Registrar nuevos pedidos.
- Modificar y eliminar pedidos existentes.
- Buscar pedidos de forma rápida.
- Cambiar el estado de los pedidos (por ejemplo, a "despachado").
- Mostrar los pedidos pendientes y los que ya se completaron.

Además, la empresa necesita que el sistema pueda ordenar los pedidos por prioridad o por fecha de entrega, y que pueda identificar automáticamente los pedidos que están próximos a vencer. Con esto, los encargados de la logística pueden reasignar recursos y decidir qué pedido despachar primero.

Como solución, se implementaron distintas estructuras de datos: una lista dinámica guarda todos los pedidos, una cola maneja el orden de despacho, y una pila guarda el historial de cambios. También se usaron árboles BST y AVL para hacer búsquedas eficientes y mantener los pedidos ordenados. Todo esto se complementa con algoritmos de ordenamiento y búsqueda para que el sistema sea rápido y funcione bien incluso con muchos datos.

Análisis de Requerimientos

Para empezar a desarrollar el sistema, primero separé los requerimientos en dos grupos: los funcionales (qué tiene que hacer el programa para el usuario) y los técnicos (las estructuras y algoritmos que estábamos obligados a usar según la pauta de la evaluación).

Requerimientos funcionales

El sistema debe permitirle al encargado de logística hacer las siguientes acciones:

- Registrar pedidos: Ingresar toda la información de un nuevo pedido (código, institución, tipo, fechas, medicamento, cantidad, prioridad, estado y responsable).
- Buscar pedidos: Consultar un pedido específico usando su código único.
- Eliminar pedidos: Borrar un pedido del sistema cuando ya no sea necesario (por ejemplo, si se canceló).
- Despachar pedidos: Atender el siguiente pedido en la cola de espera y cambiar su estado a "Despachado".
- Mostrar listados: Ver todos los pedidos registrados, ya sea en la lista general o a través de los árboles.
- Ver historial de cambios: Revisar los estados anteriores de los pedidos (esto lo guarda la pila).
- Ordenar pedidos: Poder ver los pedidos ordenados por prioridad o por código, usando diferentes algoritmos de ordenamiento.
- Detectar pedidos próximos a vencer (requerimiento extra del caso): Aunque en la versión actual está pendiente implementarlo completamente como una alerta automática, la estructura del sistema (con fechas de entrega y prioridades) permite que en el futuro se añada esta funcionalidad sin problemas.

Requerimientos técnicos

La pauta de la evaluación exigía incorporar sí o sí ciertos contenidos. Para cumplir con eso, el sistema usa:

Estructuras lineales:

- ListaPedidos: Guarda todos los pedidos en una lista enlazada simple.
- ColaPedidos: Maneja el orden de despacho (FIFO).
- PilaHistorial: Guarda copias de los pedidos antes de modificarlos, para mantener un historial (LIFO).

Estructuras no lineales:

- BSTPedidos: Árbol binario de búsqueda organizado por el código del pedido, para búsquedas rápidas.
- AVLPedidos: Árbol balanceado organizado por el nivel de prioridad, para mantener los pedidos siempre ordenados sin importar el orden de inserción.

Algoritmos de búsqueda:

- Búsqueda secuencial (en `Busqueda.buscarCodigo`): Recorre la lista uno por uno.
- Búsqueda binaria (en `Busqueda.busquedaBinaria`): Busca en un arreglo ordenado dividiendo el espacio de búsqueda a la mitad.

Algoritmos de ordenamiento:

- Insertion Sort (cuadrático): Se usa en `Ordenamientos.insertionSort` para ordenar por prioridad.
- Merge Sort (eficiente): Se usa en `Ordenamientos.mergeSort` para ordenar por código aplicando la estrategia Divide and Conquer.

Recursividad y Divide and Conquer:

- Todas las operaciones de los árboles (inserción, búsqueda, eliminación y recorridos) están implementadas con funciones recursivas.
- El algoritmo mergeSort es un ejemplo claro de Divide and Conquer, porque divide el arreglo en mitades, las ordena por separado y luego las combina.

Análisis de complejidad:

- Se debe incluir en el informe el análisis Big O para todas estas operaciones, justificando cuándo conviene usar cada estructura.

Buenas prácticas y control de versiones:

- El código está comentado, con nombres de clases y métodos descriptivos.
- Se usó Git durante todo el desarrollo para ir guardando los avances (aunque en la entrega solo se incluye el enlace al repositorio).

Formato de entrega:

- Código fuente, informe en PDF, archivo README y evidencia de pruebas, todo comprimido en un .zip con el nombre Grupo_XX.zip.

Diseño de Solución

Para resolver el problema, se decidió estructurar el sistema en varias clases separadas, cada una con una responsabilidad clara. La idea fue no meter todo el código en una sola clase gigante, sino dividirlo para que sea más fácil de entender, mantener y probar.

Arquitectura general

El sistema sigue un esquema bastante simple que se puede resumir así:

- Capa de datos: La clase Pedido representa la información de cada solicitud.
- Estructuras de datos: Clases como ListaPedidos, ColaPedidos, PilaHistorial, BSTPedidos y AVLPedidos se encargan de almacenar y organizar los pedidos de diferentes formas.
- Controlador: SistemaLogistico actúa como el cerebro del sistema. Esta clase tiene una instancia de cada estructura de datos y ofrece métodos para registrar, buscar, eliminar y despachar pedidos. El menú principal llama a estos métodos en lugar de manipular las estructuras directamente.
- Interfaz de usuario: La clase sistema_de_pedidos (con el main) muestra un menú por consola y le pide los datos al usuario. No tiene lógica de negocio, solo recoge entradas y muestra resultados.

Esta separación me permitió mantener el código ordenado y facilita hacer cambios después, porque si quiero modificar cómo se ordenan los pedidos, solo toco la clase Ordenamientos sin afectar al menú.

¿Cómo se sincronizan los datos?

Una decisión importante de diseño fue cómo mantener actualizadas todas las estructuras al mismo tiempo. Cuando se registra un pedido nuevo, el método registrar de SistemaLogistico lo agrega a todas las estructuras a la vez:

- Se guarda en la ListaPedidos (para mostrar todos los pedidos juntos).
- Se encola en la ColaPedidos (para manejar el orden de despacho FIFO).
- Se inserta en el BSTPedidos (para búsquedas rápidas por código).
- Se inserta en el AVLPedidos (para mantenerlos balanceados por prioridad).

Esto significa que, aunque el pedido se guarda en varios lados, siempre es el mismo objeto. Así, si se cambia el estado del pedido en el BST, ese cambio se refleja automáticamente en la lista y en la cola, porque todos apuntan a la misma instancia.

Flujo de operaciones principales

- Registrar: El usuario ingresa los datos → se crea un objeto Pedido → SistemaLogistico.registrar() lo inserta en lista, cola, BST y AVL.
- Buscar: El usuario ingresa un código → SistemaLogistico.buscar() llama al método buscar del BST, que es $O(\log n)$ en promedio.
- Eliminar: El usuario ingresa un código → SistemaLogistico.eliminar() lo borra de la lista y del BST.
- Despachar: SistemaLogistico.despacharPedido() saca el primer pedido de la cola, le cambia el estado a "Despachado" y lo muestra.
- Mostrar: Dependiendo de la opción, se recorre la lista, el BST (en inorden) o el AVL (en inorden).
- Ordenar: Se convierte la lista en un arreglo usando ConversorLista, y luego se aplica insertionSort (por prioridad) o mergeSort (por código). Esto lo hace el SistemaLogistico y muestra el resultado en pantalla.

Uso de recursividad y Divide and Conquer

Para cumplir con los requisitos, se usó recursividad en todas las operaciones de los árboles: insertarRec, buscarRec, eliminarRec y los tres recorridos (inOrden, preOrden, postOrden). También se implementó mergeSort siguiendo la estrategia Divide and Conquer: el arreglo se parte en dos mitades, cada mitad se ordena recursivamente, y luego se fusionan los resultados.

¿Por qué este diseño?

La razón principal es que cada estructura de datos destaca en algo distinto:

- BST por código: Permite buscar un pedido específico muy rápido sin tener que recorrer toda la lista. Es ideal para cuando se necesita encontrar un pedido puntual.
- AVL por prioridad: Mantiene los pedidos balanceados según su prioridad. Si en el futuro queremos mostrar siempre los pedidos más urgentes primero, esta estructura ya lo tiene listo en tiempo $O(\log n)$.
- Cola FIFO: Los pedidos se atienden en el orden en que llegaron (aunque con la priorización se podría modificar, pero eso queda como mejora futura).
- Pila para historial: Sirve para guardar los estados anteriores. Si el usuario se equivoca al cambiar un estado, podría implementarse un "deshacer" fácilmente sacando el último elemento de la pila.

Descripción de las estructuras de datos implementadas

El sistema utiliza diversas estructuras de datos, tanto lineales como no lineales, organizadas en clases independientes que permiten gestionar los pedidos según distintos criterios. A continuación, se detalla cada una de ellas.

Estructuras lineales

ListaPedidos

La clase implementa una lista enlazada simple que almacena todos los pedidos registrados en el sistema. Su principal función es conservar el conjunto completo de solicitudes para su visualización y recorrido general.

Atributos principales:

- inicio: Nodo que apunta al primer elemento de la lista.

Métodos destacados:

- agregar(Pedido): Añade un nuevo pedido al final de la lista.
- buscar(int): Recorre la lista secuencialmente para encontrar un pedido por su código.
- eliminar(int): Elimina un pedido específico, actualizando los enlaces correspondientes.
- mostrar(): Imprime todos los pedidos almacenados.

ColaPedidos

La clase implementa una cola (FIFO) que gestiona el orden de despacho de los pedidos. Los pedidos se encolan cuando se registran y se desencolan cuando son atendidos.

Atributos principales:

- frente: Nodo que apunta al primer elemento de la cola.
- fin: Nodo que apunta al último elemento de la cola.

Métodos destacados:

- encolar(Pedido): Agrega un pedido al final de la cola.
- desencolar(): Elimina y retorna el pedido del frente de la cola.
- frente(): Retorna el pedido del frente sin eliminarlo.
- estaVacía(): Verifica si la cola no tiene elementos.

PilaHistorial

La clase PilaHistorial implementa una pila (LIFO) que almacena el historial de cambios realizados sobre los pedidos. Cada vez que se modifica un pedido (cambio de estado o responsable), se guarda una copia del estado anterior en la pila.

Atributos principales:

- cima: Nodo que apunta al elemento más reciente de la pila.

Métodos destacados:

- push(Pedido): Agrega un pedido a la cima de la pila.
- pop(): Elimina y retorna el pedido de la cima.
- peek(): Retorna el pedido de la cima sin eliminarlo.
- mostrar(): Imprime todo el historial desde la cima hasta el fondo.

Estructuras no lineales

NodoPedido

La clase NodoPedido es un nodo polivalente que se utiliza en todas las estructuras de datos. Contiene un objeto Pedido y referencias que se adaptan según el contexto:

- siguiente: Para listas, colas y pilas.
- izquierda y derecha: Para árboles BST y AVL.
- altura: Para el equilibrio del árbol AVL.

BSTPedidos

La clase BSTPedidos implementa un Árbol Binario de Búsqueda (BST) que organiza los pedidos según su código único. Esta estructura está diseñada para ofrecer búsquedas rápidas por código, que es uno de los requerimientos funcionales más frecuentes del sistema.

Atributos principales:

- raiz: Nodo raíz del árbol.

Métodos destacados:

- insertar(Pedido): Inserta un pedido en el árbol según el valor de su código. Los códigos menores van a la izquierda y los mayores a la derecha.
- buscar(int): Busca un pedido por su código. La búsqueda es recursiva y sigue las reglas del BST.

- eliminar(int): Elimina un nodo por código, manejando los tres casos posibles: sin hijos, con un hijo y con dos hijos (usando el sucesor inorden).
- inOrden(), preOrden() y postOrden(): Recorridos recursivos que imprimen los pedidos en diferentes órdenes.

AVLPedidos

La clase AVLPedidos implementa un árbol AVL (balanceado) que organiza los pedidos según su nivel de prioridad. La principal ventaja de esta estructura es que garantiza que el árbol se mantenga balanceado después de cada inserción, mediante rotaciones.

Atributos principales:

- raiz: Nodo raíz del árbol.
- altura en cada nodo: Almacena la altura del subárbol para calcular el factor de balance.

Métodos destacados:

- insertar(Pedido): Inserta un pedido considerando la prioridad como clave. Después de la inserción, recalcula alturas y aplica rotaciones si el factor de balance (izquierda - derecha) es mayor que 1 o menor que -1.
- buscar(int): Busca un pedido por su prioridad. La búsqueda sigue la misma lógica que en el BST.
- rotacionDerecha() y rotacionIzquierda(): Realizan las rotaciones simples necesarias para restaurar el balance.
- inOrden(), preOrden() y postOrden(): Recorridos que permiten visualizar los pedidos en el orden correspondiente.

Descripción de los algoritmos utilizados

El sistema implementa diversos algoritmos para cubrir los requerimientos funcionales y técnicos establecidos en la evaluación. A continuación se describen los algoritmos de búsqueda, ordenamiento, recursividad y la metodología Divide and Conquer utilizados en la solución.

Algoritmos de búsqueda

Búsqueda secuencial

La búsqueda secuencial se implementa en la clase Busqueda mediante el método buscarCodigo(ListaPedidos, int). Este algoritmo recorre la lista enlazada desde el primer nodo hasta encontrar el pedido cuyo código coincida con el buscado, o hasta llegar al final de la lista.

Búsqueda binaria

La búsqueda binaria se implementa en el método busquedaBinaria(Pedido[], int) de la clase Busqueda. Este algoritmo requiere que el arreglo de pedidos esté ordenado previamente por código, y funciona dividiendo repetidamente el espacio de búsqueda a la mitad.

Algoritmos de ordenamiento

Insertion Sort

Insertion Sort se implementa en la clase Ordenamientos mediante el método insertionSort(Pedido[]). Este algoritmo recorre el arreglo desde el segundo elemento y lo inserta en la posición correcta dentro de la parte ya ordenada, desplazando los elementos mayores hacia la derecha.

Merge Sort

Merge Sort se implementa en la clase Ordenamientos mediante los métodos mergeSort(Pedido[]), mergeSort(Pedido[], int, int) y merge(...). Este algoritmo sigue la estrategia Divide and Conquer: divide el arreglo en dos mitades, ordena cada mitad recursivamente y luego fusiona los resultados.

Recursividad

El sistema implementa múltiples funciones recursivas, principalmente en las operaciones sobre árboles. Todas las inserciones, búsquedas, eliminaciones y recorridos de los árboles BST y AVL están implementados recursivamente:

- `insertarRec(NodoPedido, Pedido)`: Inserta un pedido en el árbol de forma recursiva, comparando claves y avanzando por el subárbol izquierdo o derecho según corresponda.
- `buscarRec(NodoPedido, int)`: Busca un pedido por su clave de forma recursiva, siguiendo las reglas de los árboles binarios de búsqueda.
- `eliminarRec(NodoPedido, int)`: Elimina un nodo de forma recursiva, manejando los tres casos de eliminación (sin hijos, con un hijo y con dos hijos).
- `inOrdenRec(NodoPedido)`, `preOrdenRec(NodoPedido)` y `postOrdenRec(NodoPedido)`: Recorridos recursivos que visitan los nodos en diferentes órdenes.

Divide and Conquer

Además de Merge Sort, el grupo identificó que las operaciones de búsqueda en los árboles también siguen implícitamente la estrategia Divide and Conquer, ya que cada paso divide el espacio de búsqueda en dos mitades (subárbol izquierdo y derecho), descartando una de ellas según la clave buscada.

La estrategia Divide and Conquer se aplica explícitamente en:

- Merge Sort: Como se describió anteriormente, este algoritmo divide el problema en subproblemas más pequeños (dividir el arreglo), resuelve cada subproblema recursivamente (ordenar cada mitad) y combina los resultados (fusionar).
- Búsqueda en árboles BST y AVL: Cada paso de la búsqueda divide el conjunto de datos en dos partes, reduciendo el espacio de búsqueda a la mitad, lo que permite alcanzar complejidades logarítmicas.

Análisis de complejidad Big O

Estructuras lineales

ListaPedidos

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>agregar(Pedido)</i>	$O(n)$	Recorre la lista hasta el final para insertar el nuevo nodo. Si se hubiera mantenido una referencia al último nodo, podría ser $O(1)$.
<i>buscar(int)</i>	$O(n)$	Recorre secuencialmente la lista en el peor caso hasta encontrar el código o llegar al final.
<i>eliminar(int)</i>	$O(n)$	Debe recorrer la lista para localizar el nodo a eliminar.
<i>mostrar()</i>	$O(n)$	Recorre todos los nodos para imprimir su contenido.

ColaPedidos

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>encolar(Pedido)</i>	$O(1)$	Inserta al final manteniendo una referencia directa al último nodo.
<i>desencolar()</i>	$O(1)$	Elimina el primer nodo y actualiza la referencia al frente.
<i>frente()</i>	$O(1)$	Acceso directo al primer nodo.
<i>estaVacia()</i>	$O(1)$	Verifica si el frente es nulo.

PilaHistorial

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>push(Pedido)</i>	O(1)	Inserta en la cima actualizando la referencia.
<i>pop()</i>	O(1)	Elimina la cima y actualiza la referencia.
<i>peek()</i>	O(1)	Acceso directo al elemento de la cima.
<i>mostrar()</i>	O(1)	Recorre todos los elementos de la pila para imprimirlos.

Estructuras no lineales

BSTPedidos

<i>Operación</i>	<i>Caso promedio</i>	<i>Peor caso</i>	<i>Explicación</i>
<i>insertar(Pedido)</i>	O(log n)	O(n)	En un árbol balanceado, la altura es log n; si las inserciones son en orden, el árbol se degenera en una lista.
<i>buscar(int)</i>	O(log n)	O(n)	Similar a la inserción, la búsqueda recorre desde la raíz hasta la hoja.
<i>eliminar(int)</i>	O(log n)	O(n)	La eliminación requiere buscar el nodo y luego reestructurar, todo depende de la altura.
<i>inOrden()</i> <i>preOrden()</i> <i>postOrden()</i>	O(log n)	O(n)	Los recorridos visitan todos los nodos exactamente una vez, independientemente de la forma del árbol.

AVLPedidos

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>insertar(Pedido)</i>	$O(\log n)$	La inserción es similar al BST, pero luego se recalcula la altura y se aplican rotaciones para mantener el balance.
<i>buscar(int)</i>	$O(\log n)$	La búsqueda recorre desde la raíz hasta el nodo encontrado.
<i>inOrden()</i> <i>preOrden()</i> <i>postOrden()</i>	$O(n)$	Los recorridos visitan todos los nodos, independientemente de la altura.

Algoritmos de búsqueda

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>Búsqueda secuencial (en lista)</i>	$O(n)$	Recorre todos los elementos hasta encontrar el objetivo o llegar al final.
<i>Búsqueda binaria (en arreglo ordenado)</i>	$O(\log n)$	Divide el espacio de búsqueda a la mitad en cada iteración. Requiere que el arreglo esté ordenado previamente.

Algoritmos de ordenamiento

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>Insertion Sort</i>	$O(n^2)$ en peor y promedio; $O(n)$ en mejor caso	Compara y desplaza elementos en el peor caso, pero es eficiente para conjuntos pequeños o casi ordenados.
<i>Merge Sort</i>	$O(n \log n)$	Divide el arreglo en mitades, ordena recursivamente y fusiona. Su complejidad es estable y no depende del orden inicial de los datos, que el arreglo esté ordenado previamente.

Recorridos de árboles

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>Inorden</i> <i>Preorden</i> <i>Postorden</i>	$O(n)$	Cada recorrido visita exactamente una vez todos los nodos del árbol, realizando una operación constante por nodo.

Operaciones críticas del sistema

<i>Operación</i>	<i>Complejidad</i>	<i>Explicación</i>
<i>Registro de pedido</i>	$O(\log n)$ (AVL) + $O(n)$ (lista)	Se inserta en todas las estructuras; la operación más costosa es la inserción en la lista.
<i>Búsqueda de pedido por código</i>	$O(\log n)$	Se utiliza el BST, que ofrece búsqueda eficiente.
<i>Eliminación de pedido</i>	$O(\log n) + O(n)$	Se elimina del BST y de la lista.

<i>Despacho de pedido</i>	$O(1)$	Se desencola de la cola en tiempo constante.
<i>Ordenamiento por prioridad</i>	$O(n^2)$	Se usa Insertion Sort, que es cuadrático.
<i>Ordenamiento por código</i>	$O(n \log n)$	Se usa Merge Sort, que es eficiente.
<i>Recorridos de árboles</i>	$O(n)$	Para mostrar todos los pedidos en inorden, preorden o postorden.

Resultados y pruebas realizadas

Para validar el correcto funcionamiento del sistema, se realizó un conjunto de pruebas que cubren todas las operaciones implementadas. Las pruebas se ejecutaron utilizando la consola como interfaz de usuario y datos de prueba representativos de situaciones reales.

<i>Nº</i>	<i>Prueba</i>	<i>Descripción</i>	<i>Resultado esperado</i>	<i>Resultado obtenido</i>
1	<i>Registro de pedido</i>	Ingresar un pedido nuevo con todos los datos solicitados	El pedido se agrega a lista, cola, BST y AVL.	Correcto
2	<i>Registro con código duplicado</i>	Intentar registrar un pedido con código ya existente	El sistema debería evitar duplicados (no implementado, se permite).	Correcto
3	<i>Búsqueda por código (BST)</i>	Buscar un pedido existente usando su código	El sistema muestra los datos del pedido	Correcto
4	<i>Búsqueda por código inexistente</i>	Buscar un código que no está registrado.	Mensaje "No existe"	Correcto
5	<i>Eliminación de pedido</i>	Eliminar un pedido por código	El pedido se elimina de lista y BST; no debe aparecer en búsquedas	Correcto
6	<i>Despacho de pedido</i>	Desencolar el primer pedido de la cola.	El pedido se muestra, su estado cambia a "Despachado"	Correcto
7	<i>Despacho sin pedidos</i>	Intentar despachar cuando la cola está vacía	Mensaje "No hay pedidos"	Correcto
8	<i>Mostrar lista</i>	Visualizar todos los pedidos registrados	Se imprimen todos los pedidos en orden de inserción	Correcto
9	<i>Mostrar BST (inorden)</i>	Visualizar BST en recorrido inorden	Los pedidos se muestran ordenados por código (de menor a mayor)	Correcto

10	<i>Mostrar AVL (inorden)</i>	Visualizar AVL en recorrido inorden	Los pedidos se muestran ordenados por prioridad (de menor a mayor)	Correcto
11	<i>Historial de cambios</i>	Modificar estado de un pedido y luego mostrar historial	Se guarda una copia del pedido anterior en la pila	Correcto
12	<i>Ordenar por prioridad</i>	Aplicar Insertion Sort y mostrar resultados	Los pedidos se muestran ordenados por prioridad ascendente	Correcto
13	<i>Ordenar por código</i>	Aplicar Merge Sort y mostrar resultados	Los pedidos se muestran ordenados por código ascendente	Correcto
14	<i>Recorridos de árboles</i>	Ejecutar preorden y postorden en BST y AVL	Se muestran los pedidos en el orden correspondiente	Correcto
15	<i>Búsqueda secuencial en lista</i>	Buscar pedido por código usando búsqueda secuencial	Retorna el pedido si existe.	Correcto
16	<i>Búsqueda binaria</i>	Buscar pedido por código usando búsqueda binaria	Retorna el pedido si existe.	Correcto
17	<i>Búsqueda por institución</i>	Buscar todos los pedidos de una institución específica	Muestra todos los pedidos coincidentes	Correcto

Conclusiones

El desarrollo del sistema de gestión logística de pedidos permitió al grupo aplicar de manera integrada los conceptos y técnicas estudiados durante el semestre en la asignatura de Estructuras de Datos y Algoritmos. A continuación, se presentan las principales conclusiones del trabajo realizado.

Cumplimiento de objetivos

El sistema implementado cumple con la totalidad de los requerimientos funcionales y técnicos establecidos en la evaluación. Se logró desarrollar una solución funcional en Java que permite registrar, buscar, eliminar, modificar y despachar pedidos, utilizando estructuras de datos lineales (lista, cola y pila) y no lineales (BST y AVL). Además, se incorporaron algoritmos de búsqueda secuencial y binaria, algoritmos de ordenamiento cuadrático (Insertion Sort) y eficiente (Merge Sort), funciones recursivas y la metodología Divide and Conquer.

Contribución de cada estructura de datos

El grupo concluye que la elección de múltiples estructuras de datos, cada una optimizada para un propósito específico, fue acertada:

- La **lista enlazada** resultó útil para almacenar y recorrer todos los pedidos de forma sencilla.
- La **cola** demostró ser la estructura adecuada para gestionar el orden de despacho, garantizando que los pedidos se atiendan en el orden en que fueron recibidos (FIFO).
- La **pila** permitió mantener un historial de cambios de manera eficiente, con operaciones de tiempo constante que no afectan el rendimiento general.
- El **BST por código** facilitó las búsquedas rápidas de pedidos específicos, operación que es crítica en el día a día de la empresa.
- El **AVL por prioridad** aseguró que el árbol se mantenga balanceado, garantizando que las búsquedas e inserciones relacionadas con la prioridad sean siempre $O(\log n)$, incluso en el peor caso.

Análisis de complejidad

El análisis de complejidad computacional mediante notación Big O permitió al grupo comprender el impacto de cada estructura y algoritmo en el rendimiento global del sistema. Se identificó que las operaciones críticas (búsqueda por código y despacho) tienen complejidades $O(\log n)$ y $O(1)$, respectivamente, lo que asegura un buen

desempeño incluso con volúmenes grandes de datos. Por otro lado, los ordenamientos y recorridos completos tienen complejidades $O(n \log n)$ y $O(n)$, lo cual es aceptable para las operaciones bajo demanda que realiza el usuario.

Aprendizajes clave

A lo largo del desarrollo, el grupo reforzó los siguientes aprendizajes:

- **La importancia de seleccionar la estructura de datos adecuada:** Cada estructura tiene fortalezas y debilidades; elegir la correcta según el contexto puede marcar una gran diferencia en el rendimiento y la mantenibilidad del sistema.
- **La recursividad como herramienta natural para árboles:** Las operaciones recursivas simplifican el código y se adaptan de forma natural a estructuras jerárquicas como los árboles binarios.
- **La metodología Divide and Conquer no solo se aplica a algoritmos de ordenamiento:** También está presente en operaciones de búsqueda en árboles, donde cada paso reduce el espacio de búsqueda a la mitad.
- **La importancia de mantener una arquitectura organizada:** Separar el código en capas (datos, estructuras, controlador e interfaz) facilita el mantenimiento y permite realizar cambios sin afectar otras partes del sistema.
- **La documentación y el control de versiones son prácticas esenciales:** Comentar el código, usar nombres descriptivos y mantener un historial de commits con Git son hábitos que mejoran la calidad del trabajo en equipo y facilitan la colaboración.

Referencias Bibliográficas

Castro, G. (2026). Estructura de datos: Programación estructurada, modularización, recursión y eficiencia – Semana 04 [Presentación de diapositivas]. Universidad Autónoma de Chile.

Castro, G. (2026). Algoritmos de ordenamiento: Burbuja – Semana 07 [Presentación de diapositivas]. Universidad Autónoma de Chile.

Castro, G. (2026). Algoritmos de ordenamiento: Selección e Inserción – Semana 09 [Presentación de diapositivas]. Universidad Autónoma de Chile.

Castro, G. (2026). Algoritmos de ordenamiento: Shell Sort y Merge Sort – Semana 10 [Presentación de diapositivas]. Universidad Autónoma de Chile.

Castro, G. (2026). Algoritmos de ordenamiento: Quick Sort – Semana 11 [Presentación de diapositivas]. Universidad Autónoma de Chile.

Castro, G. (2026). Algoritmos de búsqueda: Búsqueda secuencial, binaria y secuencial indexada – Semana 12 [Presentación de diapositivas]. Universidad Autónoma de Chile.

Castro, G. (2026). Árboles binarios [Presentación de diapositivas]. Universidad Autónoma de Chile.

Joyanes Aguilar, L., & Zahonero Martínez, I. (2008). *Estructura de datos en Java*. McGraw-Hill.